

University of Toronto

CSC490

A4

Name	Student Number
Yusuf Moola	1007658777
Mikhail Skazhenyuk	1009376337
Rudraksh Monga	1008018342
Codi Lee	1007698608

1 Background Research on GRPO

Table 1: *Literature review on visual reasoning models and their metrics to integrate with RLVR framework.*

Paper	Description	Commentary
LlamaV-o1: Rethinking Step-by-step Visual Reasoning in LLMs 5	(1) Emphasizes evaluating visual reasoning models on their step-wise reasoning process relative to a ground truth rather than just the end result or general logical consistency not tied to a ground-truth. Provides a benchmark for generic visual reasoning. (2) Provides a suite of 10 natural language metrics to evaluate VLMs on logical consistency and correctness. (3) Trains LlamaV-o1 with beam-search and CoT.	Gives an overview of useful metrics on evaluating VLMs for their reasoning process. While our simple supervised task may mostly emphasize the final result, process metrics may reveal areas for improvement in our problem structure. Metrics can be used for RLVR.
Seg-Zero: Reasoning-Chain Guided Segmentation Via Cognitive Reinforcement 4	One of the first papers that applies the GRPO approach for advanced reasoning-based segmentation. The paper shows that using just RL can improve generalization where SFT struggles. The paper tests also runs a series of ablation studies testing different RL rewards and hyperparameters.	The reward consists of format rewards and task-specific IoU rewards. Although the logical quality of reasoning itself was not tested for, it seems that enforcing the model to include a thinking section in the response improves performance. This is very simple to apply. Also, it found that non-strict IoU rewards (rather than rounded/thresholded IoU rewards helped improve generalization. Notably, this formulation does not seem to be generalized for multi-segment prompts.
LENS: Learning to Segment Anything with Unified Reinforcement Learning 2	This paper integrates VLM with SAM, and trains with GRPO to get a generalized reasoning segmentation model.	Applies VLM and RLVR with the segmentation problem, which is very close to our logo detection objective. While our goals will not need the precise segmentation unit, this paper provides a working formalism for a unified reward function for segmentation, which we can integrate as a metric for RLVR.
Few-Shot Vision-Language Reasoning for Satellite Imagery via Verifiable Rewards 1	Argues the effectiveness of few-shot RLVR for fine-tuning on domain-specific tasks.	Uses quantized IoU (intersection over union) loss for box-drawing "reasoning" tasks (called "visual grounding" tasks). Given our small dataset size and domain-specificity of the f1 logo identification task, the few-shot RLVR training pipeline linked might transfer well to our problem domain.
ViSurf: Visual Supervised-and-Reinforcement Fine-Tuning for Large Vision-and-Language Models 3	Builds on RLVR framework by integrating SFT steps into the RLVR objective function, and finds training stability and generalization robustness that exceeds RLVR and SFT performance on their own or in sequence.	This work is compelling if expecting out-of-distribution queries at inference time. It is unclear if new logos unseen during training might be considered as out-of-distribution, but this approach may be suitable if so. The paper also notices that ViSurf reduces prompt engineering requirements for RLVR. This can greatly speed up development time for structure-requiring tasks such as our own (e.g. BBOX structure).

2 Metrics and Evaluations

2.1 Quantitative (Non-VLM Friendly)

1 IoU $\rightarrow$ mAP: Mean average precision is an aggregate avg metric that uses (threshold rounded) individual intersection over union per input and the predicted and ground-truth class labels. IoU is the avg ratio ($[0,1]$) between the intersection and union of the predicted and ground-truth grounding boxes.

- **Why do we care:** The IoU metric is critical for correctness of our model. Even if the brand is correctly identified, the location and the precise size of the detection are directly linked to the brand’s exposure at a given frame. This metric is also considered fairly challenging for VLMs learning visual reasoning 1.
- **How measure:** For calculating gradients, IoU will be used. Policy models must produce predictions for the bounding boxes of a particular logo (i.e. a list of 4 pixels). This format matches with our label ground truths. The areas of the region’s unions and intersections are hence easily calculated deterministically. This is repeated for all logos in the given image and aggregated with a geometric average. We can quantize the IoU metrics like 1 during RLVR step to stabilize learning. For evaluating a model’s overall performance, mAP50-95 will be used (i.e. mAP using a 0.95 threshold for IoU to determine a positive case).
- **At scale:** The IoU metric is simple to calculate and can be vectorized, hence works well at scale. It is fairly simple to encourage VLMs to produce the correct structure to enable IoU calculation.

2.2 Qualitative (VLM RLVR Specific)

2 VLM Response Format: A 0 or 1 metric indicating whether or not the VLM’s answer follows the required format to evaluate the response.

- **Why do we care:** The GRPO rewards typically include this formatting metric as part of the unified reward to ensure that VLM responses are able to be properly measured, and pushing models to produce properly formatted responses across the RL fine-tuning phase if not.
- **How measure:** A simple regex check can determine the format of a response. Also, if using VLMs zero-shot, the VLM might misspell or reformat brand names. A fuzzy string comparison is applied to check if the answer label is indeed in the list of possible classes.
- **At scale:** Does not vectorize as this is a CPU-bound string operation. This may add some overhead to the training pipeline.

3 Informativeness of Reasoning: A qualitative metric from 1-10 generated by a judging LLM on the degree of relevant information extracted from the input image.

- **Why we care:** The primary potential benefit of using a VLM is in its zero-shot learning capabilities where the ability to perform the multimodal task of extracting the logo information from the image as text is key. This metric would measure for the amount of logo coverage extracted in the reasoning step as well as the degree of wasteful extraneous information included.
- **How measure:** We prompt a judge LLM to evaluate the model’s response compared to the ground-truth label. The ground-truth label will be the bounding box and logo names as well as

2.3 Supplementary metrics (resource usage, meta-metrics)

4 Average inference latency per 100 inputs

- **Why we care:** While we can “use a chainsaw to trim grass” by using a VLM architecture, this is likely not required for our fairly simple classification task. We expect cost and parameter count to be correlated with inference latency, and find it important to measure exactly how much latency are we compromising for how much boost in other metrics.
- **How measure:** After models have been trained, measure the time elapsed to predict per 100 inputs. These inputs can just be from the test dataset.
- **At scale:** We are simply adding a time counter for evaluating on our test dataset, which needs to be run anyway to determine the value of the model. No additional work required.

5 Average variance of rewards across policy samples

- **Why we care:** Depending on the policy model, our approach to approximating policy sampling might vary greatly. For a VLM which is inherently stochastic, with multiple reasoning paths that can still constitute a correct answer, variance may be very large and sampling improves robustness and stability during training. However, sampling approaches for more deterministic models might have different properties when applying to the RLVR framework.
- **How measure:** Take the STD of all unified rewards of each sample per iteration of GRPO.
- **At scale:** The STD of the rewards are already calculated as part of the GRPO’s optimization function. Adding this metric adds no additional computation overhead aside from aggregating all STDs at the end.

3 Establishing a baseline with LLM

3.1 Metric

The baseline LLM will examine whether the VLM response format is correct. This metric is a fundamental quality gate to ensure that downstream regex and analysis can proceed reliably.

1. **Structure:** Does the response properly format bbox coordinates?
2. **Label Consistency:** If brand names are correct
3. **Completeness:** Whether all required fields are present

3.2 Dataset Construction

1. *Training* - 101 samples
2. *Validation* - 21 samples
3. *Test* - 24 samples

Each samples is the raw text output and whether it is a valid output (binary label, 1=correct format, 0=incorrect format). Additionally, image ID, logos, etc. is included, as well as the errors encountered through our regex validator.

3.3 Baseline Approaches

3.3.1 Qwen2.5 Classifier

A prompted LLM judge to evaluate the format correctness through its natural language processing. It receives the VLM response and verifies structural requirements based on the provided examples to the original VLM input prompt. It returns a binary classification with reasoning.

3.3.2 Logistic Regression on Embeddings

We transform the sentences and perform logistic regression on embeddings, as well as tune hyperparameters.

3.4 Results

Table 2: Performance comparison of baseline format validators on test set (N=24)

Method	Precision	Recall	F1-Score	Accuracy
LogReg + Embeddings	0.960	0.958	95.5%	95.8%
Qwen2.5-VL (7B) A10 GPU	0.188	1.0	31.6%	45.8%

3.5 Errors

3.5.1 Quantitative Analysis

For LogReg + Embeddings, only one error occured: a False Negative. It most likely occurred because of an extra space between brackets in that example.

For Qwen2.5, a lot of errors occurred. Out of the 24 samples, the LLM misclassified 13, for a total accuracy of 45.8. The errors were all one sided, 13 False Positives compared to 0 False Negatives. The primary error was a failure to validate the `brand_name` field. In every single case, the LLM’s reasoning focused only on JSON structurem while the ground truth was an invalid brand name. The LLM successfully checked if the JSON was well-formed, but failed to check the brand name. However, even if provided the brand names, the LLM also incorrectly misread the schema for 12/13 `bbox_locations`. The schema requires the bbox to be in the form `[[...]]`, but the raw text has `[...]`. The LLM incorrectly interpreted this as an array of arrays. In conclusion, the LLM’s validation logic is incomplete. If given more examples and list of supported brand names, then it would world correctly.

3.5.2 Qualitative Analysis

For LogReg + Embeddings, it was both extremely fast and accurate. However, I hypothesize that it will fail on unsupported brands.

The LLM’s only issue was incorrect False Positives where it was not strict enough for the JSON schema. If it were provided a list of accepted brand names and more examples for correct bbox arrays, it would work properly. The LLM poses a critical blindspot and consistently misreads JSON schema, but it was 100% accurate on valid items. The LLM is overconfident and superficial. It provides reasoning that focuses on trivial structure checks but the prompt given lacked accepted brand names and was limited on the number of examples to communicate the necessary nested 2D array.

4 Training Judge

4.1 Metric and Hypotheses

The reward judge was trained to approximate the per-image mean Intersection-over-Union (IoU) between Qwen-predicted logo bounding boxes and the YOLO ground-truth annotations, matched on brand identity. IoU serves as the most direct and verifiable signal of spatial detection quality. The central hypothesis was that a lightweight policy network can learn consistent heuristics from Qwen’s raw detection outputs such as the number of boxes, their relative areas, aspect ratios, centre dispersion, pairwise overlaps, and the diversity of predicted brands that correlate strongly with the underlying IoU, without ever accessing the ground truth during inference. Modeling the judge’s predicted score as a random variable drawn from a Beta distribution over the range (0, 1) enables both bounded regression and probabilistic exploration during training. It was expected that larger hidden dimensions and moderately higher learning rates would accelerate convergence, whereas smaller policies or reduced datasets would underfit.

4.2 Training Pipeline

The full pipeline ingests data directly from Amazon S3, combining the YOLO-formatted dataset (images, labels, and `data.yaml`) with Qwen’s baseline predictions produced during the inference stage (`vlm_inference/qwen2.5.baseline/predictions/`). All artifacts are cached locally under `.cache/rl_judge_dataset` for reproducibility. Each Qwen prediction is parsed from its JSON text, normalizing both single-box and list-of-boxes formats, and paired with the corresponding ground-truth label. For every image, a feature-extraction module summarizes the geometry and composition of the predicted boxes, computing metrics such as count statistics, mean and variance of area and aspect ratio, centroid dispersion, brand entropy, and pairwise overlap ratios. These features form the observation vector for the policy.

The reward target for training is derived through brand-aware greedy IoU matching, ensuring each ground-truth box is used once and only matched to predictions of the same brand. The instantaneous reward is defined as the negative squared error between the sampled policy score and the ground-truth IoU, $r = -(a - \text{IoU})^2$, which is maximized when the policy assigns a score equal to the true IoU. The training loop orchestrates the full data download, caching, feature computation, batching, and gradient updates through a single command-line interface (`python -m vlm_test.rl_judge`), while logging detailed metrics to TensorBoard for monitoring.

4.3 Base Model

The judge is a two-layer multilayer perceptron (MLP) implemented in PyTorch, chosen for its simplicity and interpretability. The network consists of a linear projection, LayerNorm, GELU activation, and a second linear layer feeding two output heads that predict the Beta distribution parameters α and β . To guarantee valid parameters, α and β are passed through a `softplus + 1` transformation, enforcing values greater than 1. Sampling from $\text{Beta}(\alpha, \beta)$ produces continuous scores in (0, 1), ideal for probabilistic regression of IoU. This architecture is lightweight yet expressive enough to capture nonlinear dependencies among the geometric features extracted from Qwen predictions.

4.4 Reinforcement Learning Algorithm

Training follows the REINFORCE policy-gradient framework with an exponential moving-average (EMA) baseline to reduce gradient variance. At each step, the model samples a score $a \sim \text{Beta}(\alpha, \beta)$, computes the reward $r = -(a - \text{IoU})^2$, and updates its parameters to maximize the expected reward:

$$L = -E[(r - b) \log \pi(a|x)],$$

where b is the EMA baseline. The baseline is updated as $b_t = (1 - \beta)b_{t-1} + \beta r_t$ with a tunable smoothing coefficient β . This formulation avoids the need for environment rollouts or complex value functions; the reward is immediate, verifiable, and differentiable through the reparameterized Beta sampling. REINFORCE was selected because it is simple, stable for small

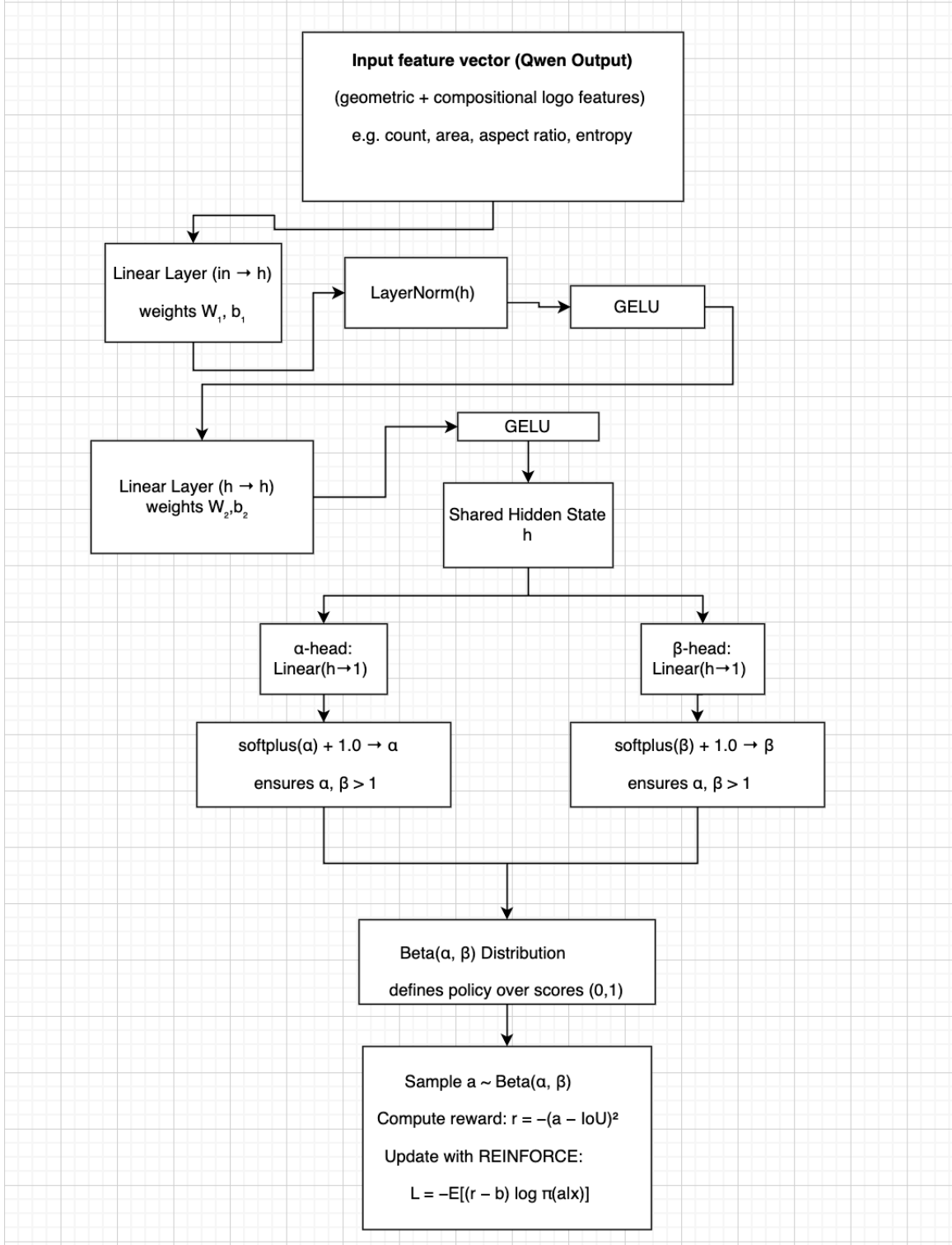


Figure 1: RL Judge architecture showing how Qwen-derived logo features are mapped through a two-layer MLP to $\text{Beta}(\alpha, \beta)$ parameters, sampling IoU-like scores and updating via REINFORCE.

policies, and well-suited to tasks where rewards can be computed directly from labeled data, unlike PPO or KL-regularized variants that require an additional reference model.

4.5 Logging and Metrics

Training statistics were captured with TensorBoard under `runs/rl_judge/<run_name>.<timestamp>`. Scalar metrics include per-step and per-epoch averages of absolute error ($|score - IoU|$), mean reward, and total loss. Histograms visualize the evolving distribution of sampled scores, confirming convergence toward realistic IoU ranges. Each run automatically logs a

JSON configuration snapshot (hyperparameters, cache paths, and device information) for reproducibility. Validation metrics are computed on a held-out 20% split by default. Example best-run curves show steadily decreasing absolute error and reward approaching 0, indicating close alignment between predicted and actual IoU.



Figure 2: RL Judge training metrics showing reward stabilization and decreasing absolute error across epochs.

4.6 Ablation Studies

Five controlled ablations were performed to evaluate learning-rate sensitivity, model capacity, batch size, baseline smoothing, and data volume. The strongest configuration with learning rate 1×10^{-3} , hidden dimension 256, and batch size 32 achieved the lowest validation absolute error (approximately 0.07) and fastest convergence. Reducing capacity (hidden dimension 64) or available data (maximum 200 samples, 40% validation split) led to clear underfitting. Smaller batches (16) increased variance in the loss curves, confirming the trade-off between gradient stability and throughput. Baseline-beta values between 0.05 and 0.1 offered similar performance; excessively small β introduced oscillations. Across runs, capacity and data quantity emerged as the primary determinants of correlation quality between predicted and true IoU.

4.7 What Did Not Work and Next Steps

Early experiments revealed JSON-parsing inconsistencies in Qwen outputs, including Markdown-fenced and non-standard bounding-box formats. These were corrected in the preprocessing layer. Lower-capacity models (hidden dimension less than or equal to 64) and limited datasets consistently underfit, plateauing at higher validation error. Reducing batch size introduced significant reward-variance noise. The current pipeline focuses solely on training the judge and does not yet apply its reward to finetune Qwen itself.

While this reinforcement learning task demonstrated that a compact policy can regress IoU with high stability, it is somewhat orthogonal to the core project goal: estimating brand visibility and return-on-investment (ROI) from Formula 1 race footage. Consequently, the next phase of work will shift focus back to end-to-end video prediction and analytics. Specifically, we plan to (1) continue fine-tuning and optimizing the YOLO detection model (2) extend inference from single frames to continuous video sequences by stitching per-frame predictions and aggregating exposure durations per brand, and (3) construct higher-level exposure metrics such as cumulative screen share, weighted by bounding-box area and visibility duration.

In parallel, future research may revisit the judge as a general reward model, incorporating both quantitative IoU and qualitative signals such as logo clarity, occlusion handling, and contextual salience. Such a hybrid judge could later be integrated into a reinforcement loop for adaptive model or prompt tuning. Expanding the dataset and performing cross-validation would further improve robustness and generalization.

Future work will extend the judge to serve as the reward function in a full RLVR loop for prompt optimization or adapter-level fine-tuning of Qwen using PPO or GRPO with KL regularization. Another planned direction is to incorporate **qualitative metrics** such as visual clarity, logo legibility, or contextual relevance in the judge’s training signal. By combining quantitative IoU-based evaluation with learned qualitative assessments, the judge could more holistically evaluate detection quality. Expanding the dataset and performing cross-validation would further strengthen robustness and generalization.

References

1. Koksai, A., and Alatan, A. (2025). *Few-Shot Vision-Language Reasoning for Satellite Imagery via Verifiable Rewards*. <https://arxiv.org/abs/2507.21745>
2. Lianghui, Z., Ouyang, B., et al. (2025). *LENS: Learning to Segment Anything with Unified Reinforced Reasoning*. <https://arxiv.org/abs/2508.14153>
3. Liu, Y., Chen, L., et al. (2025). *ViSurf: Visual Supervised-and-Reinforcement Fine-Tuning for Large Vision-and-Language Models*. <https://arxiv.org/abs/2510.10606>
4. Liu, Y., Peng, B., et al. (2025). *Seg-Zero: Reasoning-Chain Guided Segmentation via Cognitive Reinforcement*. <https://arxiv.org/abs/2503.06520>
5. Thawakar, O., Dissanayake, D. et al. (2025) *LlamaV-o1: Rethinking Step-by-step Visual Reasoning in LLMs* <https://arxiv.org/abs/2501.06186>